

LAB 2 分享

崔正平 2400010714

本次 lab 的性能优化主要从以下三个层次展开:

- 在[算法层面](#), Strassen 与 Winograd 分别减少矩阵乘法和卷积中的乘法次数.
- 在[数据组织层面](#), 通过分块、连续访问和中间结果复用改善 cache 局部性并降低内存流量.
- 在[指令执行层面](#), 针对固定问题规模展开循环, 并使用 AVX2/FMA 提高 SIMD 吞吐率和寄存器利用率.

后续两个实现都遵循相同的优化顺序: 首先调整算法结构, 然后优化数据访问, 最后设计与处理器匹配的计算内核.

Strassen 将普通分块乘法中的 8 个半尺寸乘法减少为 7 个, 因此满足

$$T(n) = 7T(n/2) + \mathcal{O}(n^2) = \mathcal{O}(n^{\log_2 7}).$$

- 乘法次数的减少会同时引入子矩阵加减、临时数据访问和递归调用开销.
- 如果递归规模过小, 这些 $\mathcal{O}(n^2)$ 复杂度的额外操作会抵消算法收益.
- 本实现在 $n \leq 64$ 时停止递归, 并切换到优化后的直接矩阵乘法.

n	相对乘法数
256	49/64
512	343/512
1024	2401/4096

- 64 阶子问题采用针对单精度浮点数设计的 4×16 AVX2/FMA 微内核.
- 微内核一次计算输出矩阵的 4 行和 16 列. 在每次 k 循环中, 连续加载 B 的两个 YMM 向量, 同时广播 A 的 4 个标量.
 - 计算过程使用 8 个 YMM 累加器执行 FMA. 部分和始终保存在寄存器中, 完成整个 64 阶归约后才写回输出矩阵.
 - 这种循环组织使同一份 B 数据被 4 个输出行复用, 并将 B 和 C 的访问转换为连续内存访问.

在递归层面, 子矩阵只使用行指针和偏移构造视图; 临时空间由一次性工作区提供, 并在深度优先递归过程中重复使用.

n	Naive (ms)	Strassen (ms)	加速比	最大相对误差
256	14.720	0.803	$18.34\times$	2.56×10^{-6}
512	111.339	4.857	$22.93\times$	5.90×10^{-6}
1024	2837.300	31.719	$89.45\times$	1.33×10^{-5}

- Strassen 从 256 阶扩展到 512 阶和 1024 阶时, 耗时分别增长 $6.05\times$ 和 $6.53\times$, 与每层产生 7 个递归子问题的结构基本一致.
- Naive 实现采用 $i-j-k$ 循环. 当 $n = 1024$ 时, 内层循环以 4 KB 步长访问 B , 产生较大的 cache 和 TLB 压力.
- 因此实验中的加速同时来自乘法次数减少、递归分块的局部性和 64 阶向量化内核.

Winograd 原理

Winograd $F(2 \times 2, 3 \times 3)$ 将一个 4×4 输入 tile 转换为一个 2×2 输出 tile:

$$U = GgG^T, \quad V = B^T dB, \quad Y = A^T \left[\sum_{c=0}^2 U_c \odot V_c \right] A.$$

实现	主要乘法次数	相对计算量
im2col + GEMM	5038848	1
Winograd	2239488	1/2.25

变换矩阵的系数只有 $0, \pm 1, \pm 0.5$, 因此三个小矩阵变换可以直接展开为固定的加减与乘 0.5, 不需要调用通用矩阵乘法。

Winograd 的乘法缩减需要通过合理的数据复用才能转化为实际性能收益.

- 首先预变换 $64 \times 3 = 192$ 个卷积核. 每个卷积核只进行一次变换, 随后用于全部输入 tile.
- 输入端预变换 $27 \times 27 \times 3 = 2187$ 个 tile. 每个变换结果在 64 个输出通道之间共享, 避免重复执行 input transform.
- 由于 $C_{in} = 3$ 为固定常数, 变换域的通道累加被直接展开. 16 个位置由两个 YMM 向量覆盖, 每 8 个位置使用一次向量乘法和两次 FMA.
- inverse transform 的 2×2 结果直接写入最终输出, 不额外保存所有输出 tile 的中间矩阵.

Winograd 结果

实现	中间数据 (float)	时间 (ms)	最大绝对误差
im2col	78732	1.167	6.10352×10^{-5}
Winograd	38064	0.407	6.10352×10^{-5}

- Winograd 的变换域中间数据为 38064 个 float, 不到 im2col column matrix 的一半.
- 在包含 kernel、input 和 output transform 的端到端计时中, Winograd 相比 im2col 获得 $2.87\times$ 加速.
- 两种实现相对 direct convolution 的最大绝对误差相同, 并且均远低于 10^{-1} 的正确性阈值.

总结

- Strassen 和 Winograd 说明, 算法复杂度或算术量的改进是获得高性能的基础, 但它并不能单独决定实际运行时间.
- 子问题规模、内存访问顺序和中间结果复用共同决定 cache 局部性以及额外开销能否被有效控制.
- 在确定数据组织之后, 固定尺寸微内核、循环展开和 AVX2/FMA 才能进一步提高处理器的执行效率.
- 本实验中, Strassen 的最高加速比为 $89.45\times$, Winograd 相比 im2col 的加速比为 $2.87\times$.

谢谢!